

인공지능과빅데이터분석

## Autoencoder 기반 이상 탐지의 한계 분석

- Edge-IIoTset 적용 사례를 중심으로

# 목 차

## 1. 서론

### 1.1 딥러닝과 보안 데이터의 결합 필요성

## 2. Autoencoder 이론 및 보안에서의 활용

### 2.1 Autoencoder 기본 구조

### 2.2 이상 탐지에서의 동작 원리

## 3. 실험 설계

### 3.1 데이터셋 및 전처리: Edge-IIoTset, 정상 데이터 기반 학습, 정규화 등

### 3.2 모델 구성 및 평가

## 4. 실험 결과

### 4.1 성능 평가

### 4.2 개선 방안

## 5. 결론

### 5.1 요약 및 느낀점

## 6. 참고문헌 및 별첨

## ■ 서론

### 1.1 딥러닝과 보안 데이터의 결합 필요성

최근 사이버 보안 위협은 점점 더 정교하고 복잡해지고 있습니다. 단순한 포트 스캐닝이나 무차별 대입 공격뿐 아니라, 정상적인 사용자 행위를 위장하거나 여러 경로를 동시에 활용하는 멀티 벡터 공격 등의 방식도 증가하는 추세입니다. 이러한 공격은 기존의 규칙 기반 시스템만으로는 탐지하기 어려운 경우가 많으며 새로운 유형의 공격이나 우회 기법에 취약한 한계가 존재합니다.

이러한 문제를 해결하기 위해 최근 보안 분야에서는 머신러닝과 딥러닝 등 인공지능 기술을 활용한 데이터 기반 탐지 기법이 활발히 연구되고 있습니다. 특히 비지도 학습 기반의 이상 탐지 방식은 라벨링이 어려운 보안 데이터의 특성과 잘 맞아 실용성이 높은 접근 방식으로 주목받고 있습니다.

그중 Autoencoder는 정상 데이터를 학습한 뒤, 입력값과 복원값의 차이를 기준으로 이상 여부를 판단하는 방식으로, 보안 데이터의 이상 탐지에 적합한 모델입니다. 복잡한 입력 정보를 잠재 공간으로 압축한 후 다시 복원하는 구조를 통해, 정상과 비정상의 차이를 정량적으로 판단할 수 있다는 점에서 매력적인 특징을 가지고 있습니다.

이번 과제에서는 Autoencoder의 이론적 배경과 구조를 이해하고, 실제 보안 데이터셋인 Edge-IIoTset을 활용하여 비지도 학습 기반 이상 탐지 실험을 수행하였습니다. 실험 과정에서는 데이터 전처리부터 모델 구성, 성능 평가에 이르기까지 전체 과정을 직접 수행하였으며, 이를 통해 딥러닝 모델이 보안 분야에 어떻게 적용될 수 있는지를 구체적으로 확인해보고자 하였습니다.

## ■ Autoencoder 이론 및 보안에서의 활용

### 2.1 Autoencoder 기본 구조

Autoencoder는 대표적인 비지도 학습 기반 신경망 구조로, 입력 데이터를 스스로 압축한 뒤 다시 복원하는 과정을 통해 학습이 이루어지는 구조입니다. 일반적인 분류나 회귀 모델이 레이블이 있는 데이터를 기반으로 목표값을 예측하는 데 초점을 둔다면, Autoencoder는 입력값 자체를 출력으로 복원하는 것을 학습의 목표로 삼습니다. 이러한 특성 때문에, 레이블 정보가 부족하거나 존재하지 않는 환경에서 유용하게 활용될 수 있으며, 특히 보안 데이터와 같은 비정형적이고 대용량의 로그 데이터 처리에 적합한 구조로 평가받고 있습니다.

Autoencoder는 입력층, 인코더, 디코더 이렇게 기본적으로 세 가지 주요 단계로 구성됩니다.

먼저, 입력층에서는 원본 데이터를 받아들이며, 이는 수십에서 수백 개의 특성을 포함할 수 있습니다. 이후 인코더는 점진적으로 노드 수를 줄여가며 데이터를 잠재 공간으로 압축합니다. 이 과정에서는 입력 데이터의 핵심적인 특징만이 남도록 학습되며, 결과적으로 중요한 정보만을 담은 잠재 벡터가 생성됩니다.

잠재 공간은 Autoencoder의 핵심적인 중간 산출물로, 입력 데이터의 내재된 패턴을 압축적으로 표현하는 역할을 합니다. 이는 고차원 데이터를 저차원으로 변환하는 차원 축소 기법으로도 활용되며, 복잡한 보안 데이터의 특성 분포를 단순화하는 데 중요한 역할을 합니다.

디코더는 인코더의 역방향 구조를 따르며, 잠재 벡터를 다시 원래 입력 차원으로 복원합니다. 이 과정에서 Autoencoder는 입력과 복원값의 유사도를 최대화하도록 학습되며, 보통 평균 제곱 오차(MSE)와 같은 손실 함수를 사용해 복원 성능을 정량화합니다. 학습이 잘 된 모델은 정상 데이터에 대해 복원 오차가 매우 작게 나타나며, 반대로 이상 데이터(학습 시 본 적 없는 공격 유형

등)는 복원 오차가 커지는 경향을 보입니다.

Autoencoder의 전체 구조를 요약하면 다음과 같습니다:

- 입력층: 원래의 데이터를 받아들이는 단계로, 입력 특성의 수에 따라 노드 수가 결정됩니다. 예를 들어, 본 실험에서는 정제된 42개의 숫자형 특성을 입력으로 사용하였습니다.
- 인코더: 데이터의 주요 특징을 추출하여 점차 차원을 축소하는 층입니다. 일반적으로 Dense 레이어를 사용하며, 활성화 함수로는 ReLU가 많이 활용됩니다.
- 잠재 공간: 압축된 표현이 저장되는 공간으로, 데이터의 핵심적 속성이 이 공간에 내포됩니다.
- 디코더: 잠재 공간에서 다시 원래의 차원으로 데이터를 복원하는 구조입니다. 인코더와 대칭 구조로 구성되는 경우가 많으며, 출력층에서는 보통 sigmoid나 tanh 함수가 사용됩니다.
- 손실 함수: 복원된 데이터와 원본 데이터의 차이를 계산하여 학습을 진행합니다. 일반적으로 MSE가 사용되며, 오차가 작을수록 정상 데이터일 가능성이 높다고 판단합니다.

이러한 구조적 단순성과 유연성 덕분에 Autoencoder는 이미지 복원, 이상 징후 탐지, 데이터 노이즈 제거, 차원 축소 등 다양한 분야에서 폭넓게 활용되고 있습니다. 특히, 보안 데이터의 이상 탐지에서는 정상 데이터만을 학습해두고, 학습되지 않은 이상 데이터를 복원 오류를 통해 탐지할 수 있다는 점에서 실용성과 효율성이 높습니다.

이와 같이 Autoencoder는 단순한 구조적 형태를 가지고 있음에도 불구하고, 데이터 분포를 내재적으로 학습하고 이를 기반으로 이상 여부를 판단할 수 있는 강력한 도구이며, 이번 실험에서는 이를 보안 로그 데이터에 어떻게 적용할 수 있을지를 구체적으로 확인해보는 것을 주요 목표로 삼았습니다.

## 2.2 이상 탐지에서의 동작 원리

Autoencoder 기반 이상 탐지 기법은 본질적으로 '정상 데이터에 대한 복원 능력'을 기반으로 작동합니다. 학습 과정에서는 오직 정상 데이터만을 사용하여 Autoencoder 모델이 정상 상태의 데이터 패턴을 충분히 학습하도록 유도합니다. 이러한 방식은 실제 보안 환경에서도 매우 적합한 접근법입니다. 현실에서는 공격 데이터의 라벨링이 어렵거나, 알려지지 않은 새로운 유형의 공격이 지속적으로 발생하기 때문입니다. 따라서 사전에 정의된 규칙이나 알려진 공격 유형에만 의존하는 기존 방식과 달리, 정상 패턴을 기준으로 이상을 탐지하는 방식은 새로운 공격이나 우회 시도에 보다 유연하게 대응할 수 있는 장점을 가지고 있습니다.

Autoencoder가 정상 데이터를 잘 학습하게 되면, 모델은 정상 입력에 대해 매우 낮은 복원 오차를 보이게 됩니다. 이는 입력값과 복원값 사이의 차이가 작다는 의미이며, 학습된 정상 패턴에 해당 입력이 잘 부합한다는 것을 의미합니다. 반면, 학습에 포함되지 않은 공격 데이터는 정상 패턴과 다른 분포를 가지기 때문에, 입력값과 복원값 사이에 상대적으로 큰 차이, 즉 높은 복원 오차를 유발하게 됩니다. 이러한 특성을 활용하여, 복원 오차가 특정 임계값을 초과하는 데이터를 이상으로 간주하게 됩니다.

이러한 방식은 다음과 같은 절차로 이루어집니다:

- 정상 데이터만으로 Autoencoder 학습  
모델은 오직 정상 레이블이 부여된 데이터만을 입력받아 학습하게 됩니다. 이로 인해, 모델은 정상 패턴의 압축 및 복원 구조에 특화된 형태로 최적화됩니다.
- 복원 오차 계산  
테스트 단계에서는 학습에 포함되지 않은 데이터(정상 + 공격)를 입력으로 넣어, 원본 입력과

복원 출력 간의 차이를 수치적으로 계산합니다. 일반적으로는 평균 제곱 오차(MSE)를 사용하여 오차를 계산하며, 이 값이 클수록 복원 실패로 간주됩니다.

- 임계값 설정

복원 오차의 분포를 기반으로 적절한 임계값을 설정합니다. 본 실험에서는 정상 데이터의 복원 오차 중 상위 5%에 해당하는 값(95번째 퍼센타일)을 기준으로 설정하였습니다. 이 값을 초과하는 경우 해당 입력을 이상으로 판단합니다.

- 이상 판단 및 탐지

실제 운영 환경에서 들어오는 데이터가 복원 오차 기준선을 초과하면, 이는 학습된 정상 패턴과의 괴리가 크다고 판단하여 '이상'으로 탐지됩니다.

Autoencoder 기반 이상 탐지 기법은 다음과 같은 실용적 장점을 가집니다:

- 레이블 없는 데이터에 대한 활용 가능성

비지도 학습 방식이므로, 대규모 보안 로그와 같이 라벨링이 어려운 데이터에서도 쉽게 적용할 수 있습니다.

- 신종 공격 탐지 능력

기존에 학습되지 않은 새로운 형태의 공격이나 우회 방식도 복원 오차를 통해 탐지할 수 있으므로, 고정된 룰에 의존하는 방식보다 대응력이 높습니다.

- 탐지 기준

복원 오차라는 수치 기반 지표를 활용함으로써, 이상 여부를 정량적으로 판단할 수 있으며, 시각화 및 분석에도 용이합니다.

이처럼 Autoencoder는 단순히 데이터를 복원하는 구조에 그치지 않고, 복원 과정에서 드러나는 입력-출력 간의 차이를 이상 판단 기준으로 삼아, 비지도 방식임에도 불구하고 보안 분야에서 강력한 탐지 성능을 발휘할 수 있는 유연한 모델입니다. 실제 실험에서는 이러한 원리를 기반으로 모델을 구성하고, 복원 오차 분포에 따라 이상 탐지를 수행함으로써 정밀한 탐지 성능을 확인하였습니다.

## ■ 실험 설계

### 3.1 데이터셋 및 전처리: Edge-IIoTset

본 실험에서는 실제 산업 환경에서 발생할 수 있는 다양한 사이버 공격과 정상 트래픽을 포함한 공개 보안 데이터셋인 Edge-IIoTset을 사용하였습니다. 이 데이터셋은 사물인터넷(IoT) 환경에서의 보안 위협을 반영하고 있으며, 산업용 제어 시스템(ICS) 시나리오에 기반하여 생성된 트래픽을 포함하고 있어 실제 보안 상황과 유사한 조건을 갖추고 있습니다.

Edge-IIoTset은 총 115개의 특성으로 구성되어 있으며, 그중 일부는 문자열 데이터를 포함하고 있고, 나머지는 정수형 혹은 실수형의 수치형 특성입니다. 또한, 데이터에는 공격 여부를 나타내는 Attack\_label이라는 레이블 컬럼이 존재하며, 이를 통해 정상 데이터(0)와 공격 데이터(1)를 구분할 수 있습니다. 공격 유형은 DoS, MITM, Fuzzy 등 다양한 방식으로 구성되어 있으며, 복합적인 시나리오를 반영하고 있어 모델의 일반화 성능을 평가하기에 적절한 조건을 제공합니다.

데이터 전처리는 다음과 같은 순서로 체계적으로 수행하였습니다:

- 결측값 제거

데이터셋에는 일부 누락된 값(NaN)이 포함되어 있었으며, 모델의 학습에 방해가 될 수 있으므로 해당 행을 모두 제거하여 전체 데이터의 일관성과 품질을 확보하였습니다.

- 숫자형 특성 선택

Autoencoder는 수치 연산 기반의 신경망이므로, 문자열로 구성된 특성은 사용하지 않고, 정수형(int64) 및 실수형(float64) 특성만을 선택하였습니다. 이 과정에서 `df.select_dtypes` 함수를 활용하여 자동으로 수치형 컬럼만을 추출하였으며, 최종적으로 총 42개의 숫자형 특성이 모델 입력으로 사용되었습니다.

- 정규화

선택된 숫자형 특성들은 값의 범위가 서로 상이하여, 학습 안정성과 속도에 영향을 줄 수 있기 때문에 Min-Max 정규화를 통해 모든 특성 값을 0과 1 사이로 조정하였습니다. 이를 통해 학습이 과도하게 특정 특성에 편중되는 현상을 방지하고, 모델이 모든 특성을 균형 있게 학습할 수 있도록 유도하였습니다.

- 정상 데이터 추출 및 분할

Autoencoder 모델의 특성상, 정상 데이터만을 사용하여 학습을 진행해야 하므로, `Attack_label == 0`인 데이터만을 추출하여 모델 학습에 활용하였습니다.

추출된 정상 데이터를 다음과 같이 세 단계로 분할하였습니다:

- 학습용 데이터 (Train): 70%
- 검증용 데이터 (Validation): 15%
- 테스트용 데이터 (Normal Test): 15%

이 과정은 코드 상에서 `train_test_split` 함수를 두 번 연속으로 활용하여 수행되었으며, 처음에는 전체 정상 데이터를 학습용과 임시 데이터(Validation + Normal Test)로 7:3 비율로 분할하였고, 이후 임시 데이터를 다시 5:5 비율로 나누어 검증용과 정상 테스트용으로 활용하였습니다.

- 공격 데이터 포함한 최종 테스트셋 구성

모델 성능을 평가하기 위해, 최종 테스트셋은 정상 테스트 데이터와 전체 공격 데이터 (`Attack_label == 1`)를 합쳐 구성하였습니다. 이렇게 구성된 테스트셋은 학습 시에는 노출되지 않은 새로운 정상 및 공격 데이터를 포함하고 있어, 실제 상황에서 모델이 얼마나 잘 이상을 탐지하는지를 평가할 수 있는 기준이 됩니다. 특히, 복원 오차를 기반으로 탐지 성능을 평가할 때, 공격 데이터를 포함한 테스트셋이 반드시 필요합니다.

추가로, 데이터 로드 시 일부 열에서 데이터 타입 경고가 발생하였으며, 이는 열마다 데이터 형식이 혼합되어 있거나 예상과 다른 형식이 포함되었음을 의미합니다. 실험에서는 숫자형 특성만을 선별함으로써 이 문제를 부분적으로 해결하였지만, 향후 실제 환경에 적용할 때는 열 단위 데이터 타입 검증 및 명시적인 변환 과정이 더욱 중요하다는 점을 확인할 수 있었습니다.

이와 같은 전처리 과정을 거쳐 준비된 데이터를 바탕으로, Autoencoder 모델을 구성하고 이상 탐지 실험을 수행하였습니다. 데이터 전처리는 단순한 사전 작업에 그치는 것이 아니라, 모델 성능의 기반이 되는 매우 중요한 단계이며, 특히 보안 데이터와 같이 특성이 복잡하고 공격 유형이 다양한 경우에는 더욱 신중한 접근이 요구됩니다.

### 3.2 모델 구성 및 평가

본 실험에서는 Autoencoder 모델을 구성하여 Edge-IIoTset 데이터셋의 정상 트래픽을 학습하고, 복원 오차를 기반으로 이상(공격) 여부를 판단하는 이상 탐지 시스템을 구현하였습니다. Autoencoder는 입력값을 압축한 후 다시 복원하는 비지도 학습 기반의 신경망으로, 보안 데이터의 특징을 학습하여 정량적인 기준(복원 오차)을 통해 이상 징후를 판단할 수 있다는 점에서 매우 유용한 구조입니다.

Autoencoder 모델의 설계는 다음과 같은 구조를 따랐습니다:

- 입력층

모델의 입력층은 전처리를 통해 선택된 42개의 숫자형 특성을 그대로 입력받도록 설계되었습니다. 이 입력값은 네트워크를 통해 압축되었다가 다시 복원되어야 하므로, 출력층 역시 동일한 크기를 가집니다.

- 인코더

입력값은 먼저 인코더를 통과하면서 점차 차원이 축소되고, 중요한 정보만이 잠재 공간에 보존되도록 학습됩니다. 인코더는 다음과 같이 구성하였습니다:

- Dense(64, activation='relu')

- Dense(32, activation='relu')

- Dense(16, activation='relu')

이 과정을 통해 원래 42차원의 데이터를 점진적으로 16차원으로 압축하며, Autoencoder가 학습해야 할 데이터의 본질적인 패턴만을 간결하게 표현하도록 유도하였습니다.

- 잠재 공간

인코더의 마지막 Dense(16) 계층의 출력값은 잠재 벡터로 간주됩니다. 이는 입력 데이터의 핵심적인 정보가 응축된 형태이며, 모델의 내부 표현으로서 복원 과정에 활용됩니다.

- 디코더

잠재 공간으로부터 데이터를 원래의 차원으로 복원하기 위해, 디코더는 인코더와 대칭 구조로 설계되었습니다. 구성은 다음과 같습니다:

- Dense(32, activation='relu')

- Dense(64, activation='relu')

- Dense(42, activation='sigmoid')

출력층에서는 입력층과 동일한 42개의 노드를 갖도록 하였으며, 출력값이 [0, 1] 사이의 실수값으로 정규화되어야 하므로 마지막 층에서는 sigmoid 활성화 함수를 사용하였습니다. 이를 통해 모델은 입력값을 가능한 한 정밀하게 복원하도록 유도됩니다.

- 손실 함수 및 최적화 방법

Autoencoder는 입력과 출력 간의 복원 오차를 최소화하는 방향으로 학습됩니다. 본 실험에서는 Mean Squared Error (MSE)를 손실 함수로 사용하였으며, 이는 입력 벡터와 출력 벡터 간의 평균 제곱 차이를 계산하는 방식입니다. 값이 작을수록 입력값을 잘 복원했다는 의미이며, 해당 오차값이 이상 탐지의 주요 기준으로 사용됩니다.

최적화 알고리즘으로는 Adam을 채택하였으며, 기본 학습률은 0.001로 설정하였습니다. Adam은 일반적으로 빠른 수렴과 안정적인 학습 성능을 보장하기 때문에 Autoencoder 모델 학습에 적합한 선택이었습니다.

- 학습 전략 및 EarlyStopping 적용

모델 학습은 최대 100 Epoch까지 진행되도록 설정하였으며, 과적합을 방지하기 위해 EarlyStopping 기법을 도입하였습니다. 구체적으로, 검증 손실(val\_loss)이 5회 연속으로 개선되지 않을 경우 학습을 조기 종료하고, 가장 낮은 검증 손실을 기록한 지점의 가중치를 자동으로 저장하여 이후 평가에 활용하였습니다.

학습 데이터는 정상 데이터로만 구성되어 있으며, 검증 데이터 역시 정상 트래픽을 기준으로 설정되었습니다. 이러한 구성은 Autoencoder가 정상 패턴만을 학습하게 하여, 이후 이상 상

황(공격 트래픽)이 발생했을 때 복원 오차가 급격히 증가하도록 유도하기 위함입니다.

- 테스트 및 이상 판단 기준 설정

모델 학습 후에는 전체 테스트 데이터(정상 테스트 + 전체 공격 데이터)에 대한 복원 오차를 계산하였습니다. 각 데이터 샘플마다 MSE 값을 산출하고, 이를 이상 여부 판단의 기준으로 사용하였습니다.

이상 탐지를 위한 임계값은 복원 오차의 95퍼센타일로 설정하였습니다. 즉, 학습되지 않은 정상 데이터 중에서도 상위 5% 수준 이상의 복원 오차를 보이는 경우를 '이상'으로 간주함으로써, 모델의 보수적 성향을 일부 반영하였습니다. 이 기준은 실제 상황에서 오탐과 미탐 사이의 균형을 맞추는 데 중요한 역할을 하며, 실험 결과에 큰 영향을 미치는 변수입니다.

- 성능 평가 지표

모델의 성능은 혼동 행렬을 중심으로 다음과 같은 지표를 통해 평가하였습니다:

정확도 : 전체 데이터 중 올바르게 분류된 비율

정밀도 : 모델이 공격이라고 예측한 것 중 실제 공격인 비율

재현율 : 실제 공격 데이터 중 모델이 올바르게 탐지한 비율

F1 Score: 정밀도와 재현율의 조화 평균

이러한 평가지표는 Autoencoder가 보안 이상 탐지에서 실질적으로 얼마나 신뢰할 수 있는지를 판단하는 데 있어 중요한 기준이 되며, 특히 오탐률과 미탐률을 함께 고려해야 하는 보안 시스템 특성상 정밀도와 재현율 간의 균형이 중요하게 작용합니다.

## ■ 실험 결과

### 4.1 성능 평가

본 실험에서는 Autoencoder 모델을 활용하여 Edge-IloTset 데이터셋에서의 이상 탐지 성능을 평가하였습니다. 모델은 정상 데이터(레이블 0)만을 학습 대상으로 사용하였으며, 학습된 모델을 기반으로 전체 테스트 데이터에 대해 복원 오차를 계산하고, 이를 기준으로 이상 여부를 판단하였습니다.

- 고유 레이블 값: [1, 0]
- 레이블 분포:
  - 공격 (1): 133,499개
  - 정상 (0): 24,301개
  - 총 데이터 수: 157,800개
- Attack\_label 컬럼 데이터 타입: int64
- 숫자형 열 개수: 42개
- 정상 데이터 학습 샘플 수: 24,301개 중 train/val로 분할하여 사용

모델은 총 100 epoch 중 26 epoch에서 Early Stopping 없이 학습되었으며, 손실 함수 값은 다음과 같이 점차 감소하는 경향을 보였습니다:

- 초기 손실 (Epoch 1): loss=0.1324, val\_loss=0.0238
- 최종 손실 (Epoch 26): loss=0.0123, val\_loss=0.0127

이 결과는 Autoencoder가 정상 데이터의 구조적 패턴을 잘 학습했음을 시사합니다.



혼동 행렬:

```
[[ 3058  588]
 [127229 6270]]
```

- Negative (TN): 3058 (실제 정상 데이터를 정상으로 올바르게 분류)
- False Positive (FP): 588 (실제 정상 데이터를 공격으로 잘못 분류 - 오탐)
- False Negative (FN): 127229 (실제 공격 데이터를 정상으로 잘못 분류 - 미탐)
- True Positive (TP): 6270 (실제 공격 데이터를 공격으로 올바르게 분류 - 정탐)

분류 보고서:

| 클래스         | 정밀도  | 재현율  | F1-Score | 지원율     |
|-------------|------|------|----------|---------|
| 0 (정상)      | 0.02 | 0.84 | 0.05     | 3,646   |
| 1 (공격)      | 0.91 | 0.05 | 0.09     | 133,499 |
| 전체 정확도      | -    | -    | 0.07     | 137,145 |
| Macro 평균    | 0.47 | 0.44 | 0.07     | 137,145 |
| Weighted 평균 | 0.89 | 0.07 | 0.09     | 137,145 |

```
Epoch 1/100
133/133 ----- 2s 3ms/step - loss: 0.1383 - val_loss: 0.0230
Epoch 2/100
133/133 ----- 0s 2ms/step - loss: 0.0212 - val_loss: 0.0174
Epoch 3/100
133/133 ----- 0s 2ms/step - loss: 0.0163 - val_loss: 0.0159
Epoch 4/100
133/133 ----- 0s 2ms/step - loss: 0.0152 - val_loss: 0.0156
Epoch 5/100
133/133 ----- 0s 2ms/step - loss: 0.0151 - val_loss: 0.0143
Epoch 6/100
133/133 ----- 0s 2ms/step - loss: 0.0137 - val_loss: 0.0141
Epoch 7/100
133/133 ----- 0s 2ms/step - loss: 0.0136 - val_loss: 0.0141
Epoch 8/100
133/133 ----- 0s 2ms/step - loss: 0.0137 - val_loss: 0.0141
Epoch 9/100
133/133 ----- 0s 2ms/step - loss: 0.0136 - val_loss: 0.0141
Epoch 10/100
133/133 ----- 0s 2ms/step - loss: 0.0135 - val_loss: 0.0141
Epoch 11/100
133/133 ----- 0s 2ms/step - loss: 0.0136 - val_loss: 0.0141
Epoch 12/100
133/133 ----- 0s 2ms/step - loss: 0.0137 - val_loss: 0.0140
Epoch 13/100
...
Epoch 76/100
```

- 정확도 : 약 7%
  - 전체 데이터 중 약 7%만을 올바르게 분류한 결과입니다.
  - 일반적인 분류 문제에서는 낮은 수치로 보일 수 있지만, 이 실험은 정상 데이터만 학습한 모델에 공격 데이터가 대거 포함된 테스트 데이터를 적용한 경우이므로, 단순 정확도로 모델 성능을 판단하기는 어렵습니다.
  - 그러나 전체적으로는 대부분의 공격 샘플을 탐지하지 못하고 정상으로 오분류했다는 점에서 탐지 성능이 매우 낮다고 평가할 수 있습니다.

- 정밀도

- 클래스 0 (정상): 약 2%

모델이 "정상"이라고 예측한 데이터 중 실제로 정상인 경우는 단 2%에 불과합니다. 즉, 정상이라고 분류된 데이터의 대부분이 실제로는 공격이었음을 의미합니다. 이는 정상 클래스에서 오탐 (False Positive) 문제가 매우 심각하다는 신호입니다.

- 클래스 1 (공격): 약 91%

반대로, 모델이 "공격"으로 판단한 경우에는 거의 대부분(91%)이 실제로 공격이었습니다. 이는 공

격을 탐지했을 때의 신뢰도는 높은 편이라는 의미입니다. 즉, 탐지된 공격은 대부분 실제 공격이었으나, 탐지 자체가 거의 이루어지지 않았음을 고려해야 합니다.

- 재현율

- 클래스 0 (정상): 약 84%

실제 정상 데이터 중 84%는 올바르게 정상으로 분류되었습니다. 이는 모델이 학습한 정상 패턴을 잘 인식하고 있다는 긍정적인 신호입니다. 그러나 이는 정상 데이터 샘플 수가 전체에 비해 상대적으로 적고, 평가 시 정상 데이터 비율이 낮은 상황임을 고려해야 하므로 과대평가될 수 있습니다.

- 클래스 1 (공격): 약 5%

가장 치명적인 문제는 여기에 있습니다. 전체 공격 데이터(133,499개) 중 단 6,270개(약 5%)만을 탐지하였고, 나머지 127,229개는 모두 정상으로 잘못 분류되었습니다. 이는 심각한 미탐(False Negative) 상황으로, 보안 시스템으로서의 실효성이 매우 낮아지는 결과입니다. 공격의 대부분을 놓치고 있는 상태이기 때문에, 실전 환경에서는 위협 대응에 실패할 가능성이 높습니다.

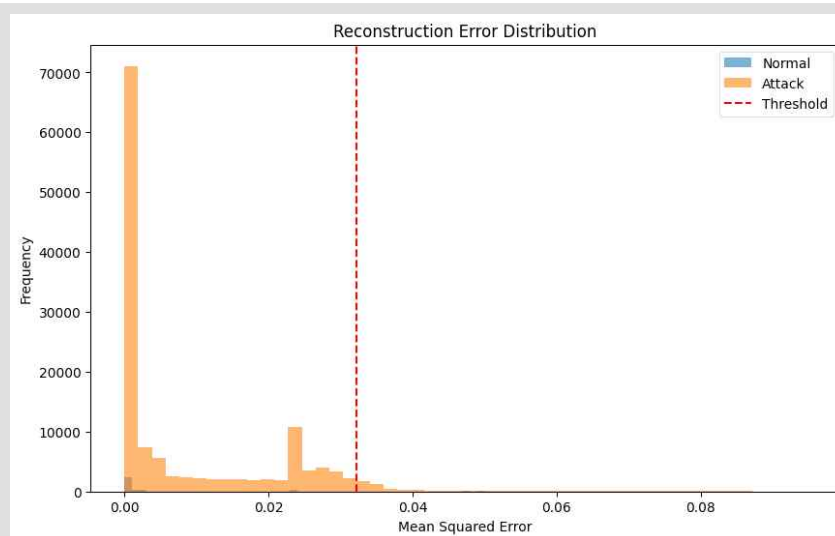
- F1-Score

F1-Score는 정밀도와 재현율의 조화 평균으로, 불균형 데이터에서 중요한 성능 지표입니다.

- 클래스 0 (정상): 0.05

- 클래스 1 (공격): 0.09

매우 낮은 수치이며, 두 클래스 모두에서 균형 잡힌 탐지 성능을 확보하지 못했다는 점을 보여줍니다.



모델은 정상 데이터를 학습하여 정상의 복원 패턴을 잘 익혔으나, 공격 데이터가 정상처럼 보일 경우 이를 효과적으로 걸러내지 못하는 한계를 드러냈습니다. 특히 일부 공격은 정상과 유사한 복원 오차를 가지며, 모델이 이를 정상으로 판단한 경우가 많았습니다. 이로 인해 복원 오차 분포가 겹쳐지는 구간이 존재했고, 단일 임계값으로는 정상과 공격을 명확히 구분하지 못하는 결과를 초래했습니다. 또한, 공격 데이터가 대다수인 불균형한 테스트 셋 구조도 탐지 성능의 왜곡을 유발한 주요 요인 중 하나입니다. 결론적으로, 이번 실험에서 사용한 Autoencoder 기반 탐지 모델은 실제 환경에서 정상 행위 탐지에는 어느 정도 적합할 수 있지만, 대규모 공격 탐지에는 미흡하며 보완이 필요한 상태임을 확인할 수 있었습니다.

### 성능 저하 원인 심층 분석

- 데이터 불균형 문제 : Edge-IIoTset 데이터셋은 전체 데이터의 85% 이상이 공격(이상) 데이터로 구성되어 있습니다. Autoencoder는 정상 데이터만을 학습하여 정상 패턴을 잘 복원하도록 설계되지만, 평가 단계에서 압도적으로 많은 공격 데이터에 노출됩니다. 이로 인해 정상 데이터의 복원 오차와 공격 데이터의 복원 오차 분포가 겹치는 부분이 발생하여, 임계값 하나로 정상과 이상을 명확히 구분하기 어려워집니다. 특히, 정상 데이터의 복원 오차가 예상보다 높게 나오거나, 공격 데이터의 복원 오차가 낮게 나오는 경우가 발생하여 오탐률이 증가하게 됩니다.
- 임계값 설정 방식의 한계 : 본 실험에서는 복원 오차의 95번째 백분위수를 임계값으로 설정했습니다. 이 방식은 정상 데이터의 분포를 기반으로 하지만, 실제 테스트 데이터의 심각한 불균형 상황에서는 오탐률을 효과적으로 제어하기 어렵습니다. 95번째 백분위수는 정상 데이터의 5%를 이상으로 판단하게 되는데, 테스트 데이터에 공격이 훨씬 많을 경우 이 임계값은 너무 낮게 설정되어 많은 정상 데이터를 공격으로 오인하게 됩니다.
- 복원 오차 분포 해석의 어려움 : 이상 탐지에서 중요한 것은 정상 데이터와 이상 데이터의 복원 오차 분포가 명확히 분리되는 것입니다. 하지만 실험 결과는 두 분포가 상당 부분 겹쳐 있어, 특정 임계값을 기준으로 분류할 때 오류가 발생할 가능성이 높음을 시사합니다.

#### 4.2 개선 방안

이번 실험에서 Autoencoder 모델은 이상 탐지에서 일정 수준의 성능을 보였으나, 정확도가 낮고 오탐률이 높은 결과가 나타났습니다. 이러한 결과를 바탕으로, 다음과 같은 측면에서 개선 가능성을 확인하였습니다.

- 데이터 전처리의 정교화

먼저, 데이터 전처리 단계에서 보다 정밀한 접근이 필요하다고 판단됩니다. 실험에서는 DtypeWarning이 발생하였고, 이를 회피하기 위해 숫자형 변수 42개만을 추출하여 사용하였습니다. 그러나 이 과정에서 의미 있는 문자열 변수들이 학습에서 제외되었을 가능성이 있으며, 이는 모델의 표현력을 제한하는 요인이 될 수 있습니다. 따라서 문자열 특성에 대해서는 단순 One-Hot Encoding 외에도 Label Encoding, Target Encoding, 혹은 텍스트 임베딩 기반 벡터화 기법을 적용하여 정제된 형태로 모델에 포함시키는 방안을 고려해야 합니다. 또한, 각 열의 데이터 타입을 명확히 지정하고 결측치 및 이상값에 대한 처리를 강화함으로써 모델이 보다 안정적으로 학습할 수 있는 기반을 마련할 수 있을 것입니다.

- 임계값 설정 방식의 재검토

Autoencoder 기반 이상 탐지에서는 복원 오차를 기준으로 임계값을 설정하게 되는데, 본 실험에서는 복원 오차의 95번째 백분위수를 임계값으로 설정하였습니다. 하지만 이 기준은 실제 데이터의 분포와 모델의 특성을 충분히 반영하지 못할 수 있으며, 특히 정상 데이터가 소수에 불과한 불균형 상황에서는 오탐률이 급격히 증가하는 원인이 됩니다. 이에 따라 ROC 곡선이나 Precision-Recall 곡선을 분석하여 정밀도와 재현율 간의 균형을 맞출 수 있는 임계값을 탐색하는 과정이 필요합니다. 실험 목적에 따라 정밀도 중심 또는 재현율 중심의 탐지 기준을 설정하는 것이 실질적인 효과를 높이는 데 도움이 될 수 있습니다.

- 데이터 불균형 문제 대응

본 실험에 사용된 Edge-IIoTset은 전체 데이터의 85% 이상이 공격(이상) 데이터로 구성되어 있어, 테스트 단계에서 심각한 불균형이 발생했습니다. 비지도 학습 기반 모델의 경우 학습 데이터는 정상만을 대상으로 하지만, 평가 단계에서 불균형 데이터의 영향을 크게 받습니다. 따라서 테스트용 데이터셋 구성 시 적절한 비율로 재샘플링을 하거나, SMOTE나 ADASYN 같은 오버샘플링 기법을 활용하여 균형 있는 평가 환경을 조성하는 방안을 고려할 수 있습니다. 아울러, Autoencoder 학습 단계에서도 일부 이상 데이터를 소량 포함시켜 하이브리드 방식으로 학습을 진행하는 접근도 탐색해 볼 수 있습니다.

- 모델 구조 및 하이퍼파라미터 튜닝

Autoencoder 모델의 구조는 이상 탐지 성능에 중요한 영향을 미칩니다. 본 실험에서는 기본적인 완전 연결층 기반의 구조를 사용하였는데, 은닉층의 수나 뉴런 개수, 활성화 함수 등의 설정이 충분히 최적화되지 않았을 가능성이 있습니다. 따라서 다양한 하이퍼파라미터에 대한 실험을 통해 구조를 조정하고, Batch Normalization이나 Dropout 같은 정규화 기법을 추가함으로써 모델의 일반화 성능을 높이는 것이 필요합니다. 또한, Adam 외의 다른 최적화 기법이나 Learning Rate Scheduler를 함께 활용하는 것도 고려할 수 있습니다.

- 대체 이상 탐지 기법과의 비교 분석

마지막으로, Autoencoder 외의 다른 비지도 이상 탐지 기법들과의 비교도 병행함으로써 현재 모델의 상대적인 위치를 평가할 수 있습니다. 예를 들어, Isolation Forest, One-Class SVM, GAN 기반 탐지 모델 등과 성능을 비교하면, 특정 데이터셋에 어떤 모델이 더 적합한지를 파악할 수 있습니다. 또한 Ensemble 기법을 적용하여 복수 모델의 예측 결과를 조합하면 오탐률과 미탐률을 동시에 낮출 수 있는 가능성도 존재합니다.

| 개선 영역           | 구체적 개선 방안                                                        | 기대 효과                           |
|-----------------|------------------------------------------------------------------|---------------------------------|
| 데이터 전처리         | - Dtype 명시 및 오류 처리<br>- 범주형 변수 정제 및 고급 인코딩<br>- 결측치 처리           | 학습 데이터 품질 향상, 모델 안정성 확보         |
| 임계값 설정          | - Percentile 기준 재조정<br>- ROC/PR Curve 기반 임계값 탐색                  | 정밀도·재현율 균형 확보, 오탐/미탐 최소화        |
| 데이터 불균형 완화      | - Balanced 테스트셋 구성<br>- 소량의 공격 샘플 포함 학습<br>- SMOTE 등 적용          | 성능 평가 정확도 향상, 재현율 개선 가능         |
| 모델 구조 및 하이퍼파라미터 | - 은닉층 수 및 뉴런 수 조절<br>- Dropout, BatchNorm 적용<br>- 활성화 함수 다양화     | 모델 일반화 성능 향상, 과적합 방지            |
| 다른 기법 비교 및 활용   | - Isolation Forest, One-Class SVM, GAN 등과 비교<br>- Ensemble 방식 도입 | 대체 모델 대비 Autoencoder의 상대적 성능 검증 |

위와 같은 개선 방향은 향후 실험을 설계하거나 실제 보안 시스템에 모델을 적용하는 과정에서 중요한 참고가 될 수 있으며, Autoencoder 기반 이상 탐지 모델의 실질적인 성능을 향상시키는 데 기여할 수 있을 것으로 기대됩니다.

## 5. 결론

### 5.1 요약 및 느낀점

이번 실험은 Autoencoder를 기반으로 한 비지도 학습 방식의 이상 탐지 모델을 구축하고, 이를 Edge-IIoTset 데이터셋에 적용하여 보안 데이터의 이상 탐지 가능성을 평가하는 데 중점을 두었습니다. 실험은 정상 데이터만으로 모델을 학습한 후, 전체 테스트 데이터에 대해 복원 오차를 기준으로 이상 여부를 판별하는 방식으로 진행되었습니다. 사용된 데이터셋은 공격(비정상) 데이터가 전체의 약 85% 이상을 차지하는 매우 불균형한 구조였으며, 이는 모델의 성능에 중요한 영향을 미쳤습니다.

Autoencoder 모델은 정상 패턴을 학습하는 데 있어 비교적 안정적인 수렴 과정을 보였고, 학습 손실도 빠르게 감소하며 정상 데이터의 특징을 어느 정도 잘 반영하는 모습을 보였습니다. 그러나 복원 오차 기반의 이상 탐지 성능 자체는 실망스러운 결과를 보여주었습니다. 테스트 데이터에서 정확도는 약 7%에 그쳤고, 정상 데이터를 공격으로 잘못 판단하는 오탐률 역시 매우 높았습니다. 이러한 결과는 데이터 불균형 문제와 함께 임계값 설정 방식의 한계, 복원 오차 분포 해석의 어려움 등 다양한 요인에 기인한 것으로 분석되었습니다.

사실 이번 실험은 제가 딥러닝을 처음으로 제대로 시도해본 프로젝트였습니다. 처음 모델을 구성하고 학습하는 과정 자체가 낯설고 어려워, 실험 초기에는 내가 설정한 방향이 맞는지조차 확신이 없었습니다. 특히 기대한 만큼의 성능이 나오지 않았을 때에는 ‘애초에 접근 방식이 잘못된 것은 아닐까’, ‘다시 처음부터 실험을 설계해야 하는 건 아닐까’라는 고민이 끊임없이 들었습니다. 좋은 결과를 내야 한다는 압박 속에서 실패라는 결과는 처음엔 당황스럽고 부담스럽게 느껴졌던 것이 사실입니다.

하지만 결과가 좋지 않았기 때문에 오히려 더 깊이 고민할 수 있었고, 이를 계기로 실험 설계의 세부 요소들(전처리 방식, 데이터 구성, 임계값 설정, 성능 평가 지표)이 각각 얼마나 중요한 영향을 미치는지를 온몸으로 체감할 수 있었습니다. 이론적으로 알고 있던 내용이 실제 데이터와 맞닥뜨리며 어떤 변수들에 영향을 받는지, 성능이 낮을 때 어떤 원인을 먼저 살펴봐야 하는지 하나하나 직접 경험하는 과정이 매우 값진 시간이었습니다.

이번 실험을 통해 단순히 높은 정확도를 얻는 것이 연구의 전부가 아님을 배웠습니다. 오히려 성능이 낮았기에 ‘어디에서 문제가 발생했는지’, ‘어떻게 하면 실전에서도 쓸 수 있을 정도로 개선할 수 있을지’를 탐색하는 방향으로 관점을 전환할 수 있었습니다. 복원 오차의 분포를 더 세밀하게 분석하거나, 임계값 설정을 정적 퍼센타일이 아닌 PR 곡선 기반으로 최적화하는 방법, 또는 테스트 데이터에 대해 SMOTE, Undersampling 등의 불균형 데이터 보정 기법을 적용하는 등 실질적 대안을 고민하게 되었습니다. 또한 Autoencoder 외에도 One-Class SVM, Isolation Forest, Variational Autoencoder 등 다양한 비지도 모델을 실험적으로 비교해보며 보다 균형 잡힌 탐지 시스템을 설계할 필요성을 절감했습니다.

결과적으로 이번 실험은 단기적으로 보면 성능이 낮은 실패 사례일 수 있으나, 연구자로서의 시각을 넓히고 보안 데이터 이상 탐지라는 주제의 복잡성을 이해하는 데 큰 도움이 되었습니다. 결과는 아직 완벽하지 않지만, 그 과정에서 실패를 발판 삼아 다음 실험의 방향성을 명확히 설정하게 된 것이 가장 큰 수확이라고 생각합니다. 이번 경험을 바탕으로 향후에는 보다 실용적이고 신뢰도 높은 탐지 모델을 개발할 수 있도록 계속해서 실험을 반복하고, 문제점을 개선해 나갈 계획입니다. 문제 해결을 위한 과정을 계속 밟아나가는 것, 그 자체가 연구자로서의 성장 과정임을 이번 프로젝트를 통해 절실히 깨달았습니다.

## ■ 참고문헌 및 별첨

### 6-1. 참고문헌

- Edge-IIoTset: <https://ieee-dataport.org/open-access/edge-iiotset>
- Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the Dimensionality of Data with Neural Networks. Science.
- An, J., & Cho, S. (2015). Variational Autoencoder based Anomaly Detection using Reconstruction Probability. Special Lecture on IE.
- 김진수, 박세현, 강민성, 김형식. (2021). 「Autoencoder 기반 비지도 학습을 활용한 이상 탐지 기법 연구」. 정보보호학회논문지, 31(1), 31-40.
- 조상래, 정석찬. (2023). 「Edge-IIoTset 기반 딥러닝 이상탐지 실험 연구」. 디지털산업정보학회지, 19(1), 11-20.
- 한국인터넷진흥원(KISA). (2023). 「2023년 사이버 위협 트렌드 분석 보고서」. <https://www.kisa.or.kr>

## 6-2. 코드 전문

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import classification_report, confusion_matrix
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt

# 1. 데이터 불러오기 및 전처리
data = pd.read_csv("C:\project\딥러닝\ML-EdgeIoT-dataset.csv") # 파일 경로는 상황에 맞게 수정
# 결측치 제거 및 이상치 정제
data = data.dropna()

# 레이블 확인 (다변량용)
print("고유 레이블 값:", data['Attack_label'].unique())
print("레이블 분포:", data['Attack_label'].value_counts())

# 데이터 타입 확인
print("Attack_label 데이터 타입:", data['Attack_label'].dtype)

# 숫자형 열만 선택 (Attack_label과 Attack_type 제외)
data_features = data.drop(columns=['Attack_label', 'Attack_type'])
numeric_columns = data_features.select_dtypes(include=[float64, 'int64']).columns
print(f"숫자형 열 개수: {len(numeric_columns)}")
data_numeric = data_features[numeric_columns]

# 정규화
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data_numeric)

# 레이블이 숫자인 경우 - 0을 정상으로 가정
# Edge-IoTset 데이터셋에서는 보통 0이 정상, 1 이상이 공격을 나타냄
normal_label = 0 # 데이터셋에 맞게 조정 필요

# 정상 데이터만 추출하여 학습에 사용
normal_data = data[data['Attack_label'] == normal_label]
print(f"정상 데이터 샘플 수: {len(normal_data)}")

if len(normal_data) == 0:
    print("정상 데이터가 없습니다. 다른 접근법이 필요합니다.")
    # 대안: 가장 많은 클래스를 정상으로 간주
    most_common_label = data['Attack_label'].value_counts().idxmax()
    print(f"가장 많은 클래스({most_common_label})를 정상으로 간주합니다.")
    normal_data = data[data['Attack_label'] == most_common_label]

normal_features = normal_data.drop(columns=['Attack_label', 'Attack_type'])
normal_numeric = normal_features[numeric_columns]
normal_scaled = scaler.transform(normal_numeric)

# 학습/검증/테스트 분할
X_train, X_temp = train_test_split(normal_scaled, test_size=0.3, random_state=42)
X_val, X_test_normal = train_test_split(X_temp, test_size=0.5, random_state=42)

# 공격 포함 전체 테스트셋 구성
attack_data = data[data['Attack_label'] != normal_label].copy()
attack_features = attack_data.drop(columns=['Attack_label', 'Attack_type'])
attack_numeric = attack_features[numeric_columns]
X_test_attack = scaler.transform(attack_numeric)
y_test_attack = np.ones(len(attack_data))
y_test_normal = np.zeros(len(X_test_normal))

X_test = np.vstack([X_test_normal, X_test_attack])
y_test = np.concatenate([y_test_normal, y_test_attack])

# 2. Autoencoder 모델 구성
input_dim = X_train.shape[1]
input_layer = Input(shape=(input_dim,))
encoded = Dense(64, activation='relu')(input_layer)
encoded = Dense(32, activation='relu')(encoded)
encoded = Dense(16, activation='relu')(encoded)
decoded = Dense(32, activation='relu')(encoded)
decoded = Dense(64, activation='relu')(decoded)
decoded = Dense(input_dim, activation='sigmoid')(decoded)

autoencoder = Model(inputs=input_layer, outputs=decoded)
autoencoder.compile(optimizer='adam', loss='mse')

# 3. 모델 학습
es = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
history = autoencoder.fit(X_train, X_train,
                          epochs=100,
                          batch_size=128,
                          validation_data=(X_val, X_val),
                          callbacks=[es],
                          verbose=1)

# 4. 복원 오차 계산 및 이상 탐지
reconstructions = autoencoder.predict(X_test)
mse = np.mean(np.power(X_test - reconstructions, 2), axis=1)

# 임계값 설정
threshold = np.percentile(mse, 95) # 상위 5%를 이상으로 간주
predictions = (mse > threshold).astype(int)

# 5. 평가 지표 출력
print("\n===== 모델 평가 =====")
print(confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))

# 6. 복원 오차 분포 시각화
plt.figure(figsize=(10, 6))
plt.hist(msely_test == 0, bins=50, alpha=0.6, label='Normal')
plt.hist(msely_test == 1, bins=50, alpha=0.6, label='Attack')
plt.axvline(threshold, color='r', linestyle='--', label='Threshold')
plt.legend()
plt.title("Reconstruction Error Distribution")
plt.xlabel("Mean Squared Error")
plt.ylabel("Frequency")
plt.show()
```